

คู่มือการตั้งค่าระบบ taskyapp Bot & Dashboard

ฉบับสมบูรณ์ สำหรับการนำระบบไปใช้งานจริงในบริษัท

คู่มือการตั้งค่าและใช้งานระบบ taskyapp — ฉบับสมบูรณ์

คู่มือนี้เขียนขึ้นจากข้อมูลจริงในโปรเจกต์ `nextjs-app` เพื่อให้ผู้ใช้งานสามารถทำตามได้ทุกขั้นตอนตั้งแต่เริ่มต้นจนระบบรันได้สำเร็จ

สารบัญ

1. ภาพรวมระบบ
2. สิ่งที่ต้องเตรียมก่อนเริ่ม
3. ขั้นตอนที่ 1: สร้างฐานข้อมูล Supabase
4. ขั้นตอนที่ 2: เตรียมข้อมูล Jira Cloud
5. ขั้นตอนที่ 3: สร้าง Webhook สำหรับ Google Chat
6. ขั้นตอนที่ 4: สร้าง Gemini API Key
7. ขั้นตอนที่ 5: รันสคริปต์ตั้งค่าอัตโนมัติ
8. ขั้นตอนที่ 6: อัปโหลดโค้ดขึ้น GitHub
9. ขั้นตอนที่ 7: Deploy ขึ้น Vercel
10. ขั้นตอนที่ 8: ตรวจสอบระบบและซิงค์ข้อมูลครั้งแรก
11. ขั้นตอนที่ 9: เชื่อมต่อ Google Chat Bot
12. ขั้นตอนที่ 10: เชื่อมต่อ Jira Webhook
13. ขั้นตอนที่ 11: ทดสอบระบบทั้งหมด
14. คู่มือการใช้งาน Dashboard
15. คู่มือการใช้งาน Chatbot

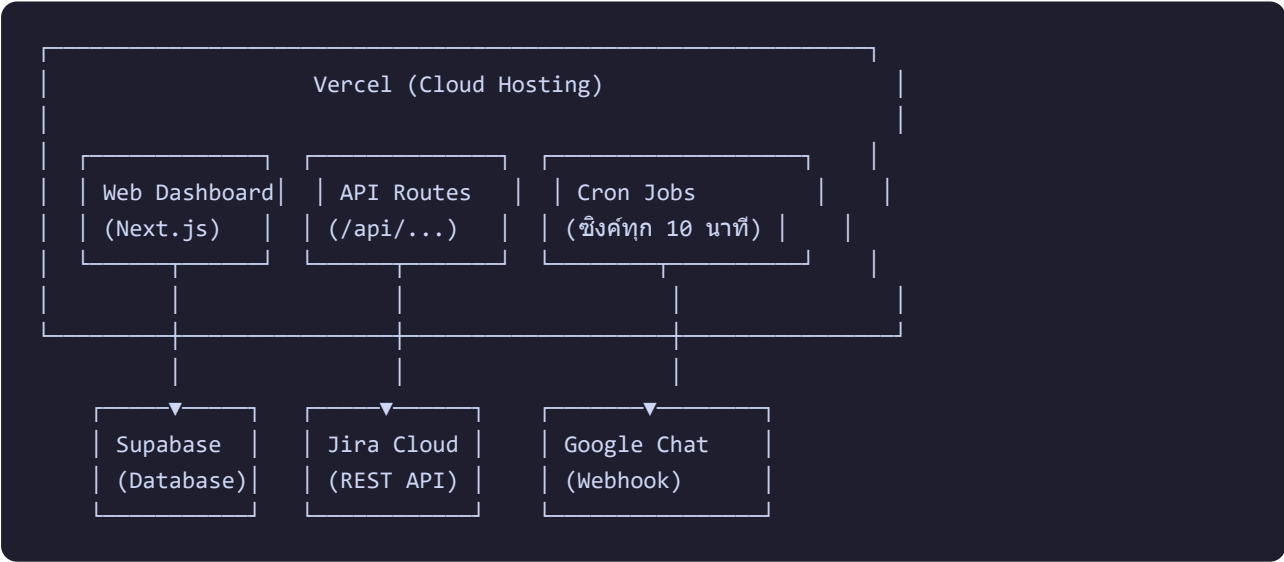
16. ตารางตัวแปรสภาพแวดล้อมทั้งหมด
17. แก้ปัญหาที่พบบ่อย (Troubleshooting)

 1. ภาพรวมระบบ taskyapp

taskyapp คือระบบจัดการงาน Jira อัจฉริยะที่ประกอบด้วย 3 ส่วนหลัก:

ส่วนประกอบ	รายละเอียด
Web Dashboard	หน้าเว็บสำหรับดูและจัดการต้งงาน มี Kanban Board, กราฟวิเคราะห์, ประวัติกิจกรรม
Google Chat Bot	บอทในห้องแชท Google Chat ที่รับคำสั่งภาษาไทยเพื่อสร้าง/แก้ไข/ค้นหางาน
Jira Sync & Notification	ระบบซิงค์ข้อมูลจาก Jira Cloud อัตโนมัติ พร้อมแจ้งเตือนทุกความเคลื่อนไหวเข้า Google Chat

สถาปัตยกรรมระบบ (System Architecture)



เทคโนโลยีที่ใช้

เทคโนโลยี	เวอร์ชัน	หน้าที่
Next.js	15.3.3	เว็บเฟรมเวิร์ค (App Router)
React	19.0.0	สร้างหน้าจอ UI
PostgreSQL (Supabase)	—	ฐานข้อมูลบนคลาวด์
Jira Cloud REST API	v3	เชื่อมต่อระบบจัดการงาน
Google Gemini AI	gemini-2.5-flash+	ประมวลผลคำสั่งภาษาธรรมชาติ
Vercel	—	โฮสต์เว็บแอปและ Cron Jobs
Recharts	2.15.3	กราฟและแผนภูมิ
Lucide React	0.511.0	ไอคอน

2. สิ่งที่ต้องเตรียมก่อนเริ่ม (Prerequisites)

TIP

เปิด Notepad หรือ Notepad++ เตรียมไว้ เพื่อจดบันทึก API Key ต่างๆ ที่จะได้ระหว่างทาง

สิ่งที่มี:

ลำดับ	สิ่งที่ต้องเตรียม	ฟรีหรือเสียเงิน	หมายเหตุ
1	คอมพิวเตอร์ที่ติดตั้ง Node.js (เวอร์ชัน 18 ขึ้นไป)	ฟรี	ดาวน์โหลดที่ https://nodejs.org
2	บัญชี GitHub	ฟรี	https://github.com
3	บัญชี Vercel	ฟรี (Hobby Plan)	https://vercel.com
4	บัญชี Supabase	ฟรี (Free Plan)	https://supabase.com
5	บัญชี Jira Cloud ขององค์กร	ฟรี (Free Plan รองรับ 10 คน)	https://www.atlassian.com/software/jira
6	บัญชี Google Workspace (สำหรับ Google Chat)	ฟรี/เสียเงิน	ต้องใช้ Google Chat แบบ Space
7	Google AI Studio (สำหรับ Gemini API Key)	ฟรี	https://aistudio.google.com
8	ไฟล์โปรเจกต์ taskyapp-handoff.zip	—	ไฟล์ที่ส่งมอบให้

ตรวจสอบว่ามี Node.js หรือยัง:

เปิด Terminal/Command Prompt แล้วพิมพ์:

```
node --version
```

ถ้าขึ้นเลขเวอร์ชัน เช่น **v18.17.0** หรือสูงกว่า แปลว่าพร้อมใช้งาน

ถ้าขึ้น Error แสดงว่ายังไม่ได้ติดตั้ง → ไปดาวน์โหลดที่ <https://nodejs.org> (เลือก LTS)

ขั้นตอนที่ 1: สร้างฐานข้อมูล Supabase

ฐานข้อมูลนี้ใช้เก็บข้อมูลตัวงาน ประวัติกิจกรรม และรายชื่อสมาชิกทีม

1.1 สมัครงและสร้างโครงการ

1. เข้า  <https://supabase.com> → กด **Start your project** → ล็อกอินด้วย GitHub
2. กด **New project**
3. กรอกข้อมูล:
 - **Name:** ตั้งชื่อโครงการ เช่น **taskyapp-db**
 - **Database Password:** ตั้งรหัสผ่าน **จดไว้ใน Notepad ห้ามลืม!**
 - **Region:** เลือก **Southeast Asia (Singapore)**
4. กด **Create new project**
5. รอประมาณ 1 นาทีจนสร้างเสร็จ

1.2 คัดลอก Database URL

1. กดปุ่ม **Connect** (สีเขียว บริเวณด้านบนสุดของหน้า)
2. เลือก **Transaction pooler** → ประเภท **URI**
3. คัดลอกลิงก์ที่ขึ้นต้นด้วย **postgresql://...**
4. สำคัญมาก! แก้ **[YOUR-PASSWORD]** ในลิงก์เป็นรหัสผ่านที่ตั้งไว้ในข้อ 1.1
5. วางลิงก์ที่แก้แล้วลง Notepad

✓ ผลลัพธ์ที่ต้องได้:

```
DATABASE_URL = postgresql://postgres.yourpassword:รหัสผ่านของคุณ@aws-0-ap-southeast-1.pooler.supabase.com
```

⚠ WARNING

ปัญหาที่พบบ่อย: ลืมแก้ **[YOUR-PASSWORD]** ถ้าคัดลอก URL มาแล้วไม่ได้แก้ **[YOUR-PASSWORD]** ระบบจะเชื่อมต่อฐานข้อมูลไม่ได้ วิธีแก้: กลับไปที่ Supabase → Connect → คัดลอก URI ใหม่ → แก้รหัสผ่านให้ถูกต้อง

⚠ WARNING

ปัญหาที่พบบ่อย: เลือกผิด **Connection type** ต้องเลือก **Transaction pooler** (ไม่ใช่ Session pooler หรือ Direct connection) ถ้าเลือกผิด Vercel จะเชื่อมต่อฐานข้อมูลไม่ได้เพราะ Serverless Functions ไม่รองรับ Direct connection

ขั้นตอนที่ 2: เตรียมข้อมูล Jira Cloud

ข้อมูลชุดนี้ใช้ให้ระบบเชื่อมต่อกับ Jira Cloud ขององค์กรเพื่ออ่าน/เขียนตั๋วงาน

2.1 หา JIRA_DOMAIN

1. ล็อกอิน Jira Cloud ขององค์กร
2. ดู URL บนแถบที่อยู่ เช่น `https://your-company.atlassian.net/...`
3. ส่วนที่อยู่ก่อน `/` คือ Domain ของคุณ เช่น `your-company.atlassian.net`
4. จดลง Notepad:

```
JIRA_DOMAIN = your-company.atlassian.net
```

2.2 หา JIRA_EMAIL

คืออีเมลที่คุณใช้ล็อกอินเข้า Jira Cloud เช่น:

```
JIRA_EMAIL = your-email@company.com
```

2.3 สร้าง JIRA_API_TOKEN

1. เข้า  `https://id.atlassian.com/manage-profile/security/api-tokens`
2. กด `Create API token`
3. ตั้งชื่อว่า `taskyapp-connector`
4. กด `Create` → คัดลอก Token ที่แสดงขึ้นมา ทันที (จะแสดงเพียงครั้งเดียว!)
5. วางลง Notepad:

```
JIRA_API_TOKEN = (Token ที่คัดลอกมา)
```

● CAUTION

ปัญหาที่พบบ่อย: คัดลอก Token ไม่ทัน Token จะแสดงเพียงครั้งเดียว ถ้าปิดหน้าต่างไปแล้วจะดูไม่ได้อีก วิธีแก้: กด `Create API token` ใหม่อีกครั้ง (Token เก่าจะยังใช้ได้อยู่ไม่ต้องลบ)

2.4 หา JIRA_PROJECT_KEY

1. เข้า Jira Cloud → เลือกโครงการที่ต้องการเชื่อมต่อ
2. ดูตัวย่อที่ปรากฏหน้า Issue เช่น `KAN-1` , `DEV-5` , `PROJ-12`
3. ตัวอักษรก่อนเครื่องหมาย `-` คือ Project Key เช่น `KAN` , `DEV` , `PROJ`
4. จดลง Notepad:

```
JIRA_PROJECT_KEY = KAN
```

✅ ผลลัพธ์ที่ต้องได้ (4 ค่า):

```
JIRA_DOMAIN      = your-company.atlassian.net
JIRA_EMAIL        = your-email@company.com
JIRA_API_TOKEN    = ATATT3xFfGF0tVuCDcZHNCpNs3HvY4ey...
JIRA_PROJECT_KEY = KAN
```

🗨️ ขั้นตอนที่ 3: สร้าง Webhook สำหรับ Google Chat

Webhook นี้ใช้ให้ระบบส่งการแจ้งเตือนเข้าห้องกลุ่ม Google Chat

1. เปิด Google Chat → คลิกเข้าห้องกลุ่ม (Space) ที่ต้องการรับการแจ้งเตือน
2. คลิกที่ **ชื่อห้อง** ด้านบนสุด → เลือก `Apps & integrations`
3. เลือก `Webhooks` → กด `Add webhook`
4. ตั้งชื่อว่า `taskyapp Notifier` → กด `Save`
5. คัดลอก URL ที่ได้ → วางลง Notepad:

```
GOOGLE_CHAT_WEBHOOK_URL = https://chat.googleapis.com/v1/spaces/xxx/messages?key=xxx&token=xxx
```

i NOTE

URL นี้จะยาวมากเป็นปกติ อย่าตัดทอนใดๆ ทั้งสิ้น

ขั้นตอนที่ 4: สร้าง Gemini API Key

API Key นี้ใช้ให้บอทเข้าใจคำสั่งภาษาไทยจากผู้ใช้ผ่าน AI

1. เข้า  <https://aistudio.google.com/apikey>
2. ล็อกอินด้วย Google Account
3. กด **Create API key**
4. เลือก Project ใดก็ได้ (หรือสร้างใหม่)
5. คัดลอก API Key ที่ได้ → วางลง Notepad:

```
GEMINI_API_KEY = AIzaSy...
```

TIP

Gemini API Key แบบฟรีรองรับได้ประมาณ 60 คำสั่งต่อนาที ซึ่งเพียงพอสำหรับการใช้งานทั่วไป

ขั้นตอนที่ 5: รันสคริปต์ตั้งค่าอัตโนมัติ

สคริปต์นี้จะนำค่าทั้งหมดที่เตรียมไว้มาสร้างไฟล์ `.env` และตั้งค่าฐานข้อมูลให้อัตโนมัติ

5.1 แยกไฟล์โปรเจกต์

1. แยกไฟล์ `taskyapp-handoff.zip` ไปยังโฟลเดอร์ที่ต้องการ
2. จะได้โครงสร้างดังนี้:


```

taskyapp-handoff/
├─ complete_setup_guide.pdf      ← คู่มือนี้ (PDF)
├─ complete_setup_guide.html    ← คู่มือนี้ (HTML)
├─ setup.bat                    ← สคริปต์สั่งค่า (Windows)
├─ setup.sh                     ← สคริปต์สั่งค่า (Mac/Linux)
├─ nextjs-app/                  ← โค้ดแอปพลิเคชันทั้งหมด
│   ├── .env.example            ← ตัวอย่างตัวแปรที่ต้องกรอก
│   ├── package.json            ← รายการ Dependencies
│   ├── setup.js                 ← ตัวช่วยตั้งค่าแบบอินเทอร์แอคทีฟ
│   ├── vercel.json              ← ตั้งค่า Cron Job
│   ├── app/                     ← หน้าเว็บและ API Routes
│   ├── components/              ← Dashboard.jsx, LoginPage.jsx
│   ├── lib/                     ← auth.js, db.js, jira.js, logs.js, nlp.js
│   ├── data/                   ← ไฟล์ข้อมูล JSON สำรอง
│   └─ public/                  ← ไฟล์ Static

```

5.2 รันสคริปต์

Windows:

1. ดับเบิลคลิกไฟล์ `setup.bat`

2. สคริปต์จะ:

- ติดตั้ง Dependencies อัตโนมัติ (`npm install`)

- เปิดตัวช่วยตั้งค่าแบบถาม-ตอบ

3. กรอกค่าที่เตรียมไว้ใน Notepad ที่ละรายการ (วางแล้วกด Enter)

Mac/Linux:

1. เปิด Terminal ในโฟลเดอร์ `taskyapp-handoff`

2. รันคำสั่ง:

```

chmod +x setup.sh
./setup.sh

```

3. กรอกค่าเหมือนกับ Windows

5.3 ค่าที่สคริปต์จะถามทีละข้อ (8 ค่า):

ลำดับ	ค่าที่ถาม	ตัวอย่าง
1	DATABASE_URL	postgresql://postgres.xxx:password@aws-...
2	JIRA_DOMAIN	your-company.atlassian.net
3	JIRA_EMAIL	your-email@company.com
4	JIRA_API_TOKEN	ATATT3xFfGF0...
5	JIRA_PROJECT_KEY	KAN
6	GOOGLE_CHAT_WEBHOOK_URL	https://chat.googleapis.com/v1/spaces/...
7	GEMINI_API_KEY	AIzaSy...
8	LLM_PROVIDER	กด Enter เลย (ค่าเริ่มต้น: <code>gemini</code>)

5.4 สิ่งที่คุณต้องทำให้อัตโนมัติ:

1. ☒ สร้างไฟล์ `.env` พร้อมค่าทั้งหมด (รวมค่าที่สร้างอัตโนมัติ: `SESSION_SECRET`, `CRON_SECRET`, `BOT_NAME`, `GEMINI_MODEL`)
2. ☒ เชื่อมต่อฐานข้อมูล Supabase
3. ☒ สร้างตารางข้อมูลทั้ง 5 ตาราง:
 - `tickets` — เก็บข้อมูลตั๋วงานทั้งหมด
 - `activity_logs` — ประวัติกิจกรรมทุกการเข้าใช้งาน
 - `team_members` — รายชื่อสมาชิกทีม
 - `action_sources` — ระบุแหล่งที่มาของการกระทำ (Dashboard/Chatbot/Jira)
 - `daily_report_log` — บันทึกการส่งรายงานประจำวัน
4. ☒ ติดตั้ง Dependencies ทั้งหมด (`npm install`)

☒ ผลลัพธ์ที่ต้องเห็นเมื่อสำเร็จ:

- ☒ สร้างไฟล์ `.env` เรียบร้อยแล้ว
- ☒ เชื่อมต่อฐานข้อมูลสำเร็จ
- ☒ สร้างตารางข้อมูลเรียบร้อย (5 ตาราง)
- ☒ ติดตั้ง Dependencies เรียบร้อย

 ตั้งค่าเสร็จสมบูรณ์!

⚠ WARNING

ปัญหาที่พบบ่อย: `npm install` ล้มเหลว อาจเกิดจาก Node.js เวอร์ชันต่ำเกินไป วิธีแก้: ตรวจสอบเวอร์ชัน Node.js ด้วย `node --version` ต้องเป็น 18.0.0 ขึ้นไป

⚠ WARNING

ปัญหาที่พบบ่อย: เชื่อมต่อฐานข้อมูลไม่ได้ สาเหตุ: `DATABASE_URL` ผิด หรือรหัสผ่านมีอักขระพิเศษ (เช่น `@`, `#`, `%`) วิธีแก้: ถ้ามหัสผ่านมีอักขระพิเศษ ให้ทำ URL Encoding เช่น `@` → `%40`, `#` → `%23`

📁 ขั้นตอนที่ 6: อัปโหลดโค้ดขึ้น GitHub

ต้องอัปโหลดโค้ดขึ้น GitHub เพื่อให้ Vercel ดึงโค้ดไป Deploy

6.1 สร้าง GitHub Repository ใหม่

1. เข้า 🖱️ `https://github.com` แล้วล็อกอิน
2. คลิกปุ่ม `+` มุมขวาบน → เลือก `New repository`
3. ตั้งชื่อ Repository เช่น `taskyapp`
4. เลือก `Private` (แนะนำ) → กดปุ่ม `Create repository`

6.2 อัปโหลดโค้ด (วิธีง่าย — ผ่านเว็บ ไม่ต้องใช้ Terminal)

1. หลังสร้าง Repo เสร็จ คลิกลิงก์ `uploading an existing file` ที่หน้า Repo
2. เปิดโฟลเดอร์ `taskyapp-handoff/nextjs-app` ในเครื่องของคุณ
3. เลือกไฟล์และโฟลเดอร์ ทั้งหมด ในนั้น แล้ว ลากวางลงในหน้า GitHub
4. เลื่อนลงล่าง กด `Commit changes` (สีเขียว)
5. รอสักครู่จนอัปโหลดเสร็จ

! IMPORTANT

อัปโหลดเฉพาะไฟล์ ข้างในโฟลเดอร์ `nextjs-app` เท่านั้น ไม่ใช่ตัวโฟลเดอร์ `nextjs-app` เอง ไม่ต้องอัปโหลด

โฟลเดอร์ `node_modules` (ถ้ามี ให้ข้ามไป)

วิธีทางเลือก (ถ้าถนัด Terminal):

```
cd taskyapp-handoff/nextjs-app
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/[ชื่อบัญชี]/[ชื่อ repo].git
git branch -M main
git push -u origin main
```

ขั้นตอนที่ 7: Deploy ขึ้น Vercel

Vercel คือบริการโฮสต์เว็บแอปที่จะรันระบบ taskyapp บนอินเทอร์เน็ต

7.1 เชื่อมต่อ Vercel กับ GitHub

1. เข้า  <https://vercel.com> → ล็อกอินด้วย GitHub
2. กด `Add New...` → `Project`
3. เลือก Repository `taskyapp` ที่เพิ่งสร้าง → กด `Import`

7.2 ตั้งค่า Root Directory

● CAUTION

ขั้นตอนนี้สำคัญมาก! ถ้าข้ามไปจะ Deploy ไม่ผ่าน

1. ในหน้า Configure Project มองหาช่อง `Root Directory`
2. กดปุ่ม `Edit` → พิมพ์ `nextjs-app` → กด Enter
3. ตรวจสอบว่าแสดงเป็น `nextjs-app` แล้ว

ทำไมต้องตั้ง Root Directory?

เพราะโค้ด Next.js อยู่ในโฟลเดอร์ย่อย `nextjs-app/` ไม่ใช่อยู่ที่ Root ของ Repository
ถ้าไม่ตั้ง Vercel จะหา `package.json` ไม่เจอและ Build ล้มเหลว

7.3 กรอก Environment Variables

1. เลื่อนลงมาที่ส่วน **Environment Variables**
2. เปิดไฟล์ `.env` ที่ได้จาก setup script (อยู่ในโฟลเดอร์ `nextjs-app/`) ด้วย Notepad
3. กด **Ctrl+A** เพื่อเลือกทั้งหมด → **Ctrl+C** คัดลอก
4. คลิกช่องแรก **Key** ในหน้า Vercel → กด **Ctrl+V** วาง
5. Vercel จะกรอก Environment Variables ทั้งหมดให้อัตโนมัติ ✓

TIP

ถ้าการวางอัตโนมัติไม่ทำงาน ให้กรอกทีละบรรทัด โดยใส่ **Key** (ชื่อตัวแปร) และ **Value** (ค่า) แยกกัน

7.4 Deploy!

1. กดปุ่ม **Deploy** (สีดำ)
2. รอประมาณ 1-2 นาที จนขึ้นว่า **Congratulations!** 🎉
3. จดลิงก์เว็บที่ได้ไว้ เช่น `https://taskapp-xxx.vercel.app` — นี่คือ "ลิงก์เว็บของคุณ" ที่จะใช้ในขั้นตอนถัดไป

WARNING

ปัญหาที่พบบ่อย: **Build ล้มเหลว (Build Error)** สาเหตุหลักๆ: 1. ลืมตั้ง **Root Directory** เป็น `nextjs-app` → กลับไปที่ Settings → General → Root Directory 2. **Environment Variables ไม่ครบ** → ไปที่ Settings → Environment Variables → ตรวจสอบว่ามีครบทุกตัว 3. **ไม่ได้อัปโหลดไฟล์ครบ** → ตรวจสอบว่า GitHub Repository มีไฟล์ `package.json`, `next.config.mjs`, โฟลเดอร์ `app/`, `lib/`, `components/`

WARNING

ปัญหาที่พบบ่อย: หลัง **Deploy สำเร็จ** แต่เปิดเว็บแล้ว **Error 500** สาเหตุ: Environment Variables ถูกตั้งค่าไม่ครบ หรือค่าผิด **วิธีแก้:** ไปที่ Vercel → Project → Settings → Environment Variables → ตรวจสอบว่ามีตัวแปรครบ 12 ตัว (ดูตารางท้ายคู่มือ) หลังแก้แล้ว กด **Redeploy** ที่หน้า Deployments

ขั้นตอนที่ 8: ตรวจสอบระบบและซิงค์ข้อมูลครั้งแรก

8.1 ตรวจสอบว่าระบบพร้อมทำงาน (Health Check)

เปิดเบราว์เซอร์ไปที่:

```
https://[ลิงก์เว็บของคุณ]/api/health
```

ตัวอย่างจริง: ถ้าลิงก์เว็บของคุณคือ `https://taskyapp-demo.vercel.app` ให้เปิด:

```
https://taskyapp-demo.vercel.app/api/health
```

ผลลัพธ์ที่ถูกต้อง:

```
{
  "overall": "ready",
  "checks": {
    "database": { "status": "connected" },
    "jira": { "status": "connected" },
    "google_chat": { "status": "configured" },
    "gemini": { "status": "configured" },
    "env_vars": { "status": "complete" }
  }
}
```

- ✅ `"overall": "ready"` → ระบบพร้อมใช้งานทุกส่วน
- ⚠️ `"overall": "needs_attention"` → มีบางส่วนที่ต้องตรวจสอบ (อ่านรายละเอียดในผลลัพธ์)

⚠️ WARNING

ปัญหาที่พบบ่อย: `database status = "error"` วิธีแก้: ตรวจสอบค่า `DATABASE_URL` ใน Vercel Environment Variables ว่าถูกต้องหรือไม่ (โดยเฉพาะรหัสผ่าน)

⚠️ WARNING

ปัญหาที่พบบ่อย: `jira status = "error"` วิธีแก้: ตรวจสอบ 4 ค่า: `JIRA_DOMAIN` , `JIRA_EMAIL` , `JIRA_API_TOKEN` , `JIRA_PROJECT_KEY` ลองเข้า Jira Cloud ด้วยอีเมลเดียวกัน ถ้าล็อกอินไม่ได้ แสดงว่าอีเมลหรือ Token ผิด

8.2 ดึงข้อมูลจาก Jira เข้าระบบครั้งแรก

เปิดเบราว์เซอร์ไปที่:

```
https://[ลิงก์เว็บของคุณ]/api/cron/sync-tickets?secret=[CRON_SECRET]
```

ตัวอย่างจริง: ถ้า `CRON_SECRET` ของคุณคือ `652ea9793ef9888681ff82f12f467a9e` :

```
https://taskyapp-demo.vercel.app/api/cron/sync-tickets?secret=652ea9793ef9888681ff82f12f467a9e
```

TIP

หาค่า `CRON_SECRET` ได้จากไฟล์ `.env` ในโฟลเดอร์ `nextjs-app/` หรือใน Vercel → Settings → Environment Variables

ผลลัพธ์ที่ถูกต้อง:

```
{"success": true, "message": "Synchronized 10 tickets successfully.", "count": 10}
```

(ตัวเลข `count` จะเป็นจำนวนตั๋วงานจริงที่อยู่ในโครงการ Jira ของคุณ)

8.3 ล็อกอินเข้า Dashboard

1. เปิดเบราว์เซอร์ไปที่ `https://[ลิงก์เว็บของคุณ]`
2. จะเห็นหน้าจอล็อกอิน
3. ใช้บัญชีทดสอบ (UAT) ตามตารางด้านล่าง:

Username	Password	ชื่อที่แสดง	บทบาท	สิทธิ์
ceo	1234	สมชาย ผู้บริหาร	CEO	เต็มสิทธิ์
pm	1234	สมหญิง จัดการ	Project Manager	เต็มสิทธิ์
admin	admin	ผู้ดูแลระบบ	Admin	เต็มสิทธิ์
dev1	1234	สมศักดิ์ พัฒนา	Developer	ดูและแก้ไข
dev2	1234	สมใจ โค้ดดิ้ง	Developer	ดูและแก้ไข
design	1234	สมศรี ออกแบบ	Designer	ดูและแก้ไขจำกัด
qa	1234	สมปอง ทดสอบ	QA Tester	ดูและแก้ไขจำกัด
sales	1234	สมฤดี ขายดี	Sales	ดูอย่างเดียว

ขั้นตอนที่ 9: เชื่อมต่อ Google Chat Bot

ขั้นตอนนี้ทำให้คุณสามารถส่งงาน Jira ผ่าน Google Chat ได้

9.1 สร้าง Google Chat App ผ่าน Apps Script

1. เข้า  <https://script.google.com> → กด **New project**
2. ตั้งชื่อโครงการว่า **taskyapp Bot**
3. ลบโค้ดเดิมทั้งหมด แล้ววางโค้ดนี้แทน:


```
function onMessage(event) {
  var url = "https://[ลิงก์เว็บของคุณ]/api/webhook";
  var payload = {
    message: {
      text: event.message.text,
      sender: {
        displayName: event.user.displayName
      },
      space: {
        name: event.space.name
      }
    }
  };

  var options = {
    method: "post",
    contentType: "application/json",
    payload: JSON.stringify(payload)
  };

  var response = UrlFetchApp.fetch(url, options);
  var result = JSON.parse(response.getContentText());

  return {
    text: result.text || "ไม่สามารถประมวลผลคำสั่งได้"
  };
}
```

4. สำคัญ! แก้ **[ลิงก์เว็บของคุณ]** ในบรรทัดที่ 2 เป็นลิงก์ Vercel ของคุณ เช่น:

```
var url = "https://taskyapp-demo.vercel.app/api/webhook";
```

9.2 Deploy เป็น Chat App

1. กด **Deploy** → **New deployment**
2. ประเภท: เลือก **Add-on** → ประเภทย่อย **Google Chat App**
3. กด **Deploy**

9.3 ตั้งค่าใน Google Cloud Console

1. เข้า  <https://console.cloud.google.com>
2. เลือก Project เดียวกับ Apps Script
3. ไปที่ **APIs & Services** → **Google Chat API** → **Configuration**
4. กรอก:

- App name: **taskyapp**
- Avatar URL: (เว้นว่างได้)
- Description: **บอทจัดการงาน Jira**
- Functionality: ☒ Receive 1:1 messages, ☒ Join spaces and group conversations
- Connection settings: เลือก **Apps Script** → วาง Deployment ID
- Visibility: เลือก Specific people หรือ Everyone in organization

5. กด Save

9.4 เพิ่มบอทเข้าห้อง Google Chat

1. เปิด Google Chat → เข้าห้องกลุ่มที่ต้องการ
2. คลิกชื่อห้อง → **Apps & integrations** → **Add apps**
3. ค้นหาชื่อ **taskyapp** → กด **Add**
4. ทดสอบโดยพิมพ์ **@taskyapp ช่วยเหลือ**

⚠ WARNING

ปัญหาที่พบบ่อย: บอทไม่ตอบ 1. ตรวจสอบว่า Deploy แล้วจริง (กด Deploy → ต้องมี Deployment ID) 2. ตรวจสอบว่า URL ใน Apps Script ถูกต้อง (ต้องลงท้ายด้วย **/api/webhook**) 3. ตรวจสอบว่า Google Chat API เปิดอยู่ใน Google Cloud Console 4. เข้า Apps Script → ดู Execution Log ว่ามี Error อะไร

🔗 ขั้นตอนที่ 10: เชื่อมต่อ Jira Webhook

Webhook นี้ทำให้ Jira ส่งสัญญาณมาหาระบบ taskyapp ทุกครั้งที่มีการสร้างหรือแก้ไขตัวงาน

10.1 ตั้งค่า Webhook ใน Jira Cloud

1. เข้า Jira Cloud → คลิก **Settings** (มุมขวาบน) → **System**
2. เลื่อนลงมาหาหัวข้อ **Webhooks** (ในส่วน Advanced) → กด **Create a Webhook**
3. กรอกข้อมูล:

ช่อง	ค่าที่ต้องการกรอก
Name	taskyapp Notifier
Status	Enabled
URL	https://[ลิงก์เว็บของคุณ]/api/jira/webhook

4. ในส่วน Events ให้เลือก:

- ☒ Issue → created
- ☒ Issue → updated

5. กด Create

☒ ผลลัพธ์ที่ต้องเห็น:

เมื่อมีคนสร้างหรือแก้ไขตัวงานใน Jira → ข้อความแจ้งเตือนจะปรากฏในห้อง Google Chat อัตโนมัติ

⚠ WARNING

ปัญหาที่พบบ่อย: แจ้งเตือนไม่เข้า Google Chat 1. ตรวจสอบ URL ของ Webhook ว่าถูกต้อง (ต้องลงท้ายด้วย /api/jira/webhook) 2. ตรวจสอบว่า Events ถูกถูก (Issue created, Issue updated) 3. ตรวจสอบว่า GOOGLE_CHAT_WEBHOOK_URL ตั้งค่าถูกต้องใน Vercel Environment Variables

i NOTE

เรื่องแจ้งเตือนแจ้ง 2 รอบ: ระบบได้รับการแก้ไขแล้วให้กรองเฉพาะฟิลด์ที่ผู้สนใจจริงเท่านั้น (status, assignee, summary, description, due date, priority, labels) และจะไม่แจ้งเตือนซ้ำสำหรับการอัปเดตระบบภายในอย่าง Rank ที่ Jira ส่งมาเมื่อลากตัวบน Kanban Board

☒ ขั้นตอนที่ 11: ทดสอบระบบทั้งหมด

ทดสอบที่ 1: สร้างงานผ่าน Chatbot

ใน Google Chat พิมพ์:

@taskyapp สร้างงาน ทดสอบระบบ taskyapp

ผลลัพธ์ที่ต้องเห็น:

1. ☒ บอตอบกลับว่าสร้างงานสำเร็จ พร้อมรหัสตัว เช่น **KAN-15**
2. ☒ ข้อความแจ้งเตือนปรากฏในห้อง Chat (จาก Jira Webhook)
3. ☒ ตัวงานใหม่ปรากฏใน Jira Cloud
4. ☒ ตัวงานใหม่ปรากฏบน Web Dashboard (อาจต้องรีเฟรช)

ทดสอบที่ 2: เปลี่ยนสถานะผ่าน Dashboard

1. เข้า Dashboard → ล็อกอินด้วย **pm** / **1234**
2. ลากตัวจากคอลัมน์ **To Do** ไป **In Progress**

ผลลัพธ์ที่ต้องเห็น:

1. ☒ สถานะเปลี่ยนใน Jira Cloud
2. ☒ ข้อความแจ้งเตือนปรากฏในห้อง Google Chat
3. ☒ แจ้งเตือนระบุชื่อผู้ดำเนินการพร้อมบทบาท เช่น **สมหญิง จัดการ (Project Manager)**

ทดสอบที่ 3: แก้ไขงานผ่าน Chatbot

@taskyapp แก้ไข KAN-15 สถานะ done

ผลลัพธ์ที่ต้องเห็น:

1. ☒ บอตอบกลับว่าแก้ไขสำเร็จ
2. ☒ สถานะเปลี่ยนใน Jira Cloud
3. ☒ แจ้งเตือนปรากฏในห้อง Chat

คู่มือการใช้งาน Dashboard

แท็บหลักทั้ง 5 แท็บ:

แท็บ	ฟีเจอร์	คำอธิบาย
 Kanban Board	ลากวาง	ดูและจัดการทำงานแบบ Kanban (To Do → In Progress → In Review → Done)
 Analytics	กราฟ/แผนภูมิ	สถิติภาพรวมโครงการ กราฟสถานะ/ลำดับความสำคัญ/ภาระงาน
 Team	จัดการทีม	เพิ่ม/ลบสมาชิกทีม เชื่อมโยงชื่อ Jira กับชื่อเล่น
 Activity Logs	ประวัติกิจกรรม	ดูทุกกิจกรรมที่เกิดขึ้น กรองตามประเภท/ค้นหาตามชื่อ
 Task Detail	รายละเอียดงาน	คลิกที่ตัวเพื่อดูรายละเอียด แก้ไขหัวข้อ/รายละเอียด/ผู้รับผิดชอบ/กำหนดส่ง

ระบบสิทธิ์ตามบทบาท (Role-Based Access):

บทบาท	ดูข้อมูล	แก้ไขงาน	สร้างงาน	เปลี่ยนสถานะ	จัดการทีม
CEO	✓	✓	✓	✓	✓
Project Manager	✓	✓	✓	✓	✓
Admin	✓	✓	✓	✓	✓
Developer	✓	✓	✓	✓	✗
Designer	✓	✓ (จำกัด)	✗	✗	✗
QA Tester	✓	✓ (จำกัด)	✗	✗	✗
Sales	✓	✗	✗	✗	✗

 คู่มือการใช้งาน Chatbot

คำสั่งทั้งหมดที่ใช้ได้:

คำสั่ง	ตัวอย่าง	คำอธิบาย
สร้างงาน	@taskyapp สร้างงาน แก้มักหน้า Login	สร้างตัวงานใหม่ใน Jira
สร้างหลายงาน	@taskyapp สร้าง ออกแบบ UI และ เขียน API	สร้างหลายตัวพร้อมกัน
แก้ไขงาน	@taskyapp แก้ไข KAN-5 สถานะ in progress	แก้ไขข้อมูลตัวที่มีอยู่
เปลี่ยนสถานะ	@taskyapp เปลี่ยน KAN-5 เป็น done	เปลี่ยนสถานะตัว
ดูงาน	@taskyapp ดู KAN-5	แสดงรายละเอียดตัว
สรุปงาน	@taskyapp สรุป	แสดงสถิติภาพรวมโครงการ
ค้นหา	@taskyapp ค้นหา login	ค้นหาตัวตามคำค้น
ช่วยเหลือ	@taskyapp ช่วยเหลือ	แสดงรายการคำสั่งทั้งหมด

ตัวอย่างคำสั่งขั้นสูง:

@taskyapp สร้างงาน ออกแบบหน้า Dashboard มอบหมายให้ สมศรี ความสำคัญ สูง กำหนดส่ง 2026-07-15

→ สร้างงานพร้อมกำหนดผู้รับผิดชอบ ลำดับความสำคัญ และวันกำหนดส่งในครั้งเดียว

ตารางตัวแปรสภาพแวดล้อมทั้งหมด

ตัวแปร	ต้อง กรอก เอง?	คำอธิบาย	ตัวอย่าง
DATABASE_URL	✓ ต้อง กรอก	ลิงก์เชื่อมต่อฐานข้อมูล Supabase	postgresql://postgres.xxx:password@...
JIRA_DOMAIN	✓ ต้อง กรอก	โดเมน Jira Cloud	your-company.atlassian.net
JIRA_EMAIL	✓ ต้อง กรอก	อีเมลที่ใช้ล็อกอิน Jira	you@company.com
JIRA_API_TOKEN	✓ ต้อง กรอก	API Token จาก Atlassian	ATATT3xFfGF0...
JIRA_PROJECT_KEY	✓ ต้อง กรอก	รหัสย่อโครงการ	KAN
GOOGLE_CHAT_WEBHOOK_URL	✓ ต้อง กรอก	URL Webhook ของ Google Chat	https://chat.googleapis.com/...
GEMINI_API_KEY	✓ ต้อง กรอก	API Key จาก Google AI Studio	AIzaSy...
LLM_PROVIDER	สคริปต์สร้างให้	ผู้ให้บริการ AI	gemini
GEMINI_MODEL	สคริปต์สร้างให้	โมเดล AI ที่ใช้	gemini-2.5-flash

ตัวแปร	ต้อง กรอก เอง?	คำอธิบาย	ตัวอย่าง
<code>BOT_NAME</code>	สคริปต์ สร้าง ให้	ชื่อบอท	<code>taskyapp</code>
<code>SESSION_SECRET</code>	สคริปต์ สร้าง ให้	คีย์เข้า รหัส Session	(สุ่มอัตโนมัติ 64 ตัว)
<code>CRON_SECRET</code>	สคริปต์ สร้าง ให้	รหัสลับ สำหรับ Cron API	(สุ่มอัตโนมัติ 32 ตัว)

แก้ปัญหาที่พบบ่อย (Troubleshooting)

❌ ปัญหา: เปิดเว็บแล้วขึ้น Error 500 (Internal Server Error)

สาเหตุ: Environment Variables ไม่ครบหรือค่าผิด

วิธีแก้:

1. เข้า Vercel → Project → Settings → Environment Variables
2. ตรวจสอบว่ามีตัวแปรครบ 12 ตัว (ดูตารางด้านบน)
3. ตรวจสอบว่า `DATABASE_URL` มีรหัสผ่านที่ถูกต้อง
4. หลังแก้แล้ว ไปที่ Deployments → กด : (สามจุด) → **Redeploy**

❌ ปัญหา: ล็อกอินเข้า Dashboard ไม่ได้

สาเหตุ: ใช้ username/password ผิด

วิธีแก้: ใช้บัญชีจากตารางในขั้นตอนที่ 8.3 เช่น `admin` / `admin` หรือ `ceo` / `1234`

✖ ปัญหา: Dashboard ไม่แสดงตัวงาน (หน้าว่าง)

สาเหตุ: ยังไม่ได้ซิงค์ข้อมูลจาก Jira ครั้งแรก

วิธีแก้:

1. เปิดเบราว์เซอร์ไปที่ URL ซิงค์ (ดูขั้นตอนที่ 8.2)
 2. ตรวจสอบว่า Jira Project มีตัวงานอยู่จริง
 3. ตรวจสอบว่า `JIRA_PROJECT_KEY` ตรงกับ Project ที่ต้องการ
-

✖ ปัญหา: บอท Google Chat ไม่ตอบ

สาเหตุที่เป็นไปได้:

1. ยังไม่ได้ Deploy Apps Script → กด Deploy → New deployment
2. URL ใน Apps Script ผิด → ตรวจสอบว่าลงท้ายด้วย `/api/webhook`
3. Google Chat API ไม่ได้เปิด → เปิดใน Google Cloud Console
4. บอทยังไม่ได้ถูกเพิ่มเข้าห้อง → เพิ่มผ่าน Apps & integrations

วิธีตรวจสอบ: เข้า Apps Script → Executions → ดู Log ว่ามี Error อะไร

✖ ปัญหา: Jira แจ้งเตือนไม่เข้า Google Chat

สาเหตุที่เป็นไปได้:

1. Jira Webhook URL ผิด → ต้องลงท้ายด้วย `/api/jira/webhook`
2. Events ไม่ได้ติ๊ก → ต้องติ๊ก Issue created และ Issue updated
3. `GOOGLE_CHAT_WEBHOOK_URL` ผิด → ตรวจสอบใน Vercel Environment Variables

วิธีตรวจสอบ:

1. สร้างหรือแก้ไขตัวใน Jira Cloud
 2. ดู Vercel Function Logs (Vercel → Project → Logs) ว่ามี request เข้ามาหรือไม่
-

❌ ปัญหา: แจ้งเตือนได้ 2 รอบเมื่อลากตัวบน Kanban Board

สาเหตุ: เมื่อลากตัวเปลี่ยนคอลัมน์ Jira จะส่ง Webhook มา 2 ครั้ง:

1. ครั้งที่ 1: อัปเดตสถานะ (เช่น To Do → In Progress)
2. ครั้งที่ 2: อัปเดตอันดับตำแหน่ง (Rank) ภายในคอลัมน์ใหม่

สถานะ: ✅ แก้ไขแล้ว — ระบบจะกรองและแจ้งเตือนเฉพาะฟิลด์ที่มีความหมายเท่านั้น (status, assignee, summary, description, due date, priority, labels) โดยจะข้ามฟิลด์ภายในอย่าง Rank ไปโดยอัตโนมัติ

❌ ปัญหา: Cron Job ไม่ทำงานอัตโนมัติ (ข้อมูลไม่ซิงค์ทุก 10 นาที)

สาเหตุ: Vercel Free Plan (Hobby) ไม่รองรับ Cron Jobs

วิธีแก้: มี 2 ทางเลือก:

1. อัปเกรดเป็น Vercel Pro (แนะนำ) → Cron Jobs จะทำงานอัตโนมัติตาม `vercel.json`
2. ใช้บริการ Cron ภายนอก เช่น <https://cron-job.org> (ฟรี) → ตั้งให้เรียก URL ซิงค์ทุก 10 นาที:

```
https://[ลิงก์เว็บของคุณ]/api/cron/sync-tickets?secret=[CRON_SECRET]
```

❌ ปัญหา: ต้องการอัปเดตโค้ดหลังจาก Deploy แล้ว

วิธีทำ:

1. แก้ไขโค้ดในเครื่อง
2. Push ขึ้น GitHub:

```
git add .
git commit -m "อัปเดตฟีเจอร์ใหม่"
git push origin main
```

3. Vercel จะ Deploy ใหม่อัตโนมัติภายใน 1-2 นาที

! IMPORTANT

ขั้นตอนทั้งหมดนี้ทำเพียงครั้งเดียว หลังจากตั้งค่าเสร็จแล้ว ระบบจะทำงานอัตโนมัติตลอดเวลา Vercel จะ Deploy ใหม่อัตโนมัติทุกครั้งที่มีการแก้ไขโค้ดใน GitHub

— จบคู่มือ —